

Design Report

• **Architecture + DB choice justification**

The system uses a monolithic Next.js App Router architecture where UI pages and backend APIs live in one codebase. This was chosen because it is simple to develop, easy to deploy with Docker, and avoids managing separate frontend/backend services.

Data is stored in SQLite through Prisma ORM. SQLite was a good fit for course scope: no external DB account, easy local/container setup, and reliable persistence for relational forum data. Prisma adds type-safe queries, schema-driven migrations, and a clean data model for User, Channel, Post, Reply, Vote, and Attachment.

The schema supports key requirements directly:

- Nested replies via parentReplyId self-relation
- One vote per user per target via compound unique key
- Cascading deletes for moderation cleanup

• **API endpoints overview**

Auth:

- POST /api/auth/signup create user with validation + hashed password
- /api/auth/[...nextauth] sign in/sign out/session

Channels:

- GET /api/channels list channels
- POST /api/channels create channel (auth required)
- DELETE /api/channels/[id] admin delete channel

Posts:

- GET /api/posts?channelId=... list posts in channel
- POST /api/posts create post (auth required)
- GET /api/posts/[id] get single post + attachments

- DELETE /api/posts/[id] admin delete post

Replies:

- GET /api/replies?postId=... list replies (oldest-first)
- POST /api/replies create reply or nested reply (parentReplyId)
- DELETE /api/replies/[id] admin delete reply

Votes:

- GET /api/votes?targetType=&targetId= read score + current user vote
- POST /api/votes set vote +1, -1, or 0 (remove)

Search:

- GET /api/search supports: posts text, replies text, author content, top/bottom user, top/bottom ranked content

Admin users:

- GET /api/users admin list users
- DELETE /api/users/[id] admin delete user (with safeguards)

Uploads/attachments:

- POST /api/uploads upload screenshot (validated)
- GET /api/uploads/[id] serve screenshot via controlled route
- GET /api/attachments?targetType=&targetId= list attachment metadata

• Screenshot storage approach

Screenshots are stored on the filesystem (uploads) and indexed in the DB (Attachment table).

Upload flow:

1. Client sends fileName, mimeType, and base64 data to POST /api/uploads
2. Server validates:
 - MIME type allowed: PNG/JPEG/WebP

- Extension allowed: .png, .jpg, .jpeg, .webp
 - Max size: 5 MB
3. File is saved with randomized filename in uploads
 4. Metadata is saved in DB (targetType, targetId, mimeType, sizeBytes, storageRef)
 5. File is served through GET /api/uploads/[id] with explicit content headers

Why this approach:

- Meets assignment constraints
- Simple and reliable in Docker/local setup
- Safer than exposing raw file execution paths
- Keeps DB lean by storing metadata instead of blobs

• Key packages + why

- next, react, react-dom: full-stack app framework and UI runtime
- next-auth: credential-based auth, sessions, sign-in/sign-out
- prisma, @prisma/client: schema-based ORM and type-safe DB access
- @prisma/adapters-better-sqlite3: Prisma adapter for SQLite datasource
- bcryptjs: password hashing (no plaintext storage)
- tailwindcss: fast, consistent responsive styling
- vitest: automated test runner for API functional tests